EMERGING WORLD MODELS · FOURTH IN THE SERIES

Attention on lattice traversal is all you need

The EWM lattice is the map, attention is the locomotion, materialization is the door between them — and a dispatcher that learns which host LLM should move.

SUPPORT = JOIN · SOFTMAX = ANNEALING · HEADS = WORLDS · LEARNED DISPATCH

ALEX MYLNIKOV · ALEXANDR MYLNIKOV JR. · DEEPENDRA KUMAR

github.com/alexmy21/ewm-state-machine



A traveler, a map, and a door

1. **The map.** A good geo map is exact about **where** things are — and silent about **what they are like inside**. Every town is a named dot; the map folds into a pocket and never forgets.

2. **The walking.** To know a town you must go in and walk — one street at a time, seeing only what is around you now. Walking is rich but local, and by itself it leaves no trace on the map.

3. **The door.** The town gate joins the two views of one place — the dot and the streets. It is **faithful**: walk out, mark what you saw on the map, and you may re-enter as often as you like.

> **This deck's claim:** the map and the walking are two halves of one journey — and around the pair, a coordinator learns **which guide should walk**.

THE STORY → THE DECK

the map

the EWM lattice — content-addressed waypoints, exact and durable

the walking

attention — graduated motion inside the detail

the door

materialization on the way in, ingestion on the way out

the guides

host LLMs — each speaks its own token language

the coordinator

the dispatcher — learns who should move

The walking, in one line — and five properties

$$\mathrm{Attn}(X) = \mathrm{softmax}\left(QK^{\mathrm{T}} / \sqrt{d} \right) \cdot V, \quad Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

1 **Content routing.** The score q·k depends on the two states alone — order enters only through position encoding.

2 **Permutation equivariance.** Shuffle the tokens and the result shuffles the same way — attention cannot tell orderings apart.

3 **Competition.** Softmax normalises each row — keys compete for one fixed unit of weight. This is the selection step.

4 **Value aggregation.** The output is a weighted combination of the values — a convex mix, nothing discarded.

5 **Composability.** Heads concatenate and project; blocks stack around a residual stream $x \leftarrow x + \mathrm{MHA}(x)$.

Two facts that matter later: softmax is strictly positive — an attention row has *no zeros*; selection is never hard.

> The walker's rule, read structurally: **content-based routing followed by monotone accumulation** — a query selects items by similarity, and their values are combined. That shape has an exact, set-valued twin.

The EWM lattice: a map of content-addressed waypoints

An **HLLSet** is a compact sketch of a token collection: each token is hashed once onto one bit of a fixed **32,768-bit plane** (4 KB packed). Ten tokens or ten million — always 4 KB. Order does not matter, repetition does not matter.

All sketches share one plane, so set operations are **exact bit operations**; every sketch is **content-addressed** (SHA-1).

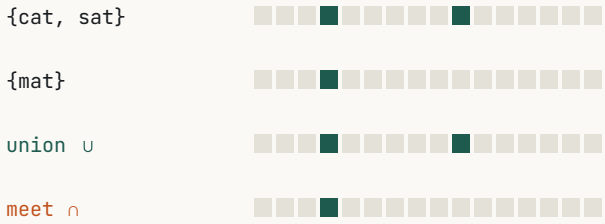
`B = ( 2^plane, ⊆, ∪, ∩, Δ )  — the map itself`

A **node is a place with its detail compressed inside**: the bitmap is the coordinate, not the transcript. A side table (the LUT) records which tokens landed on each bit — collisions are resolved there, not in the bits.

Each **world** — a hash configuration plus a threshold — reads the same lattice through its own LUT. Worlds share the plane and differ only in interpretation.

> The map answers **which place** — scored by overlap (BSS = |X∩B| / |B|), selected by a threshold, accumulated by union. What it cannot do is move inside the detail.

A TOY PLANE — 16 POSITIONS



`cat → 3, sat → 9, mat → 3`

Two tokens share bit 3 — a collision. The LUT reads bit 3 back as {cat, mat}; the bits carry the geometry, the LUT carries the meaning.

Real plane: P = 10, 1,024 registers × 32 bits = 32,768 positions.

The seam: the support of attention is a lattice join

$$\mathrm{supp}\Big(\sum_i a_i \cdot V_i\Big) = \bigcup \{\, V_i : a_i > 0 \,\}$$
$$(a_i \geq 0,\ V_i \text{ Boolean})$$

Support–union lemma. With nonnegative weights, a weighted sum of bit vectors has a non-zero entry exactly where at least one attended vector has a 1. The **bitmap of an attention layer is a node of the lattice**.

The weights never change *which* bits are kept — only the convex coordinates *inside* the kept set. The bitmap fixes the working set; the weights move within it.

And the similarity is shared too: overlap $\langle x, y \rangle = |X \cap Y|$ — EWM's BSS, Jaccard and containment are all normalisations of **the same bilinear form** attention's dot product computes.

> The seam is one operator, not a resemblance: a traversal's output can be handed to an attention layer, and an attention layer's support handed back to the lattice — **without translation loss**.

A 4-BIT PICTURE

$V_1 = 1\ 0\ 1\ 1$

$V_2 = 0\ 1\ 1\ 0$

$0.7 \cdot V_1 + 0.3 \cdot V_2$

$= .7\ .3\ 1\ .7$

$\mathrm{supp} = 1\ 1\ 1\ 1 = V_1 \cup V_2$

Every attended value contributes its bits; the weights decide how strongly — never whether.

Correspondence theorem. Thresholded attention and lattice traversal share one support operator, one similarity geometry, one accumulation law — they differ only in the interpolation inside.

Temperature and threshold are two faces of one knob

Softmax is the unique minimiser of a **free energy**:
$F(\alpha) = -\langle \alpha, \text{scores} \rangle + \tau \cdot \Sigma\, \alpha_i \ln \alpha_i$.
The first term rewards high scores; the second is an entropy that smooths the choice.

Turn the temperature down and the weights concentrate on the best-scoring

$$\alpha_i(\tau) = e^{(s_i/\tau)} \,/\, \Sigma_j\, e^{(s_j/\tau)}$$

The EWM **world threshold θ** is the attention threshold — the hard, zero-temperature selection. The softmax temperature is its **entropic relaxation**: the same rule, made differentiable.

> Attention and traversal are **two ends of one temperature axis** — and the softmax is the annealing schedule between them.

SAME SCORES, THREE TEMPERATURES

τ → 0 — the hard end

Only the argmax survives: weights (1, 0, 0). This *is* thresholded traversal — the map's selection rule.

τ = 1 — the working point

Graduated weights, e.g. (0.67, 0.24, 0.09): smooth locomotion, the bitmap of the support unchanged.

τ → ∞ — the uniform end

All keys weighted equally: (⅓, ⅓, ⅓). No selection at all — the join of everything.

An untruncated attention layer is a traversal at θ = −∞. "Hard" selection is what the threshold makes explicit — and what makes the map exact.

Materialization is the door — and it is faithful

A node is a **coordinate, not a transcript** — its detail (lexical content, structure, statistics) is compressed into the bits and the LUT. To act inside a node, the model needs tokens again.

Materialization restores an HLLSet context into a token context; **ingestion** is the same door walked the other way.

```
tokens → ingest → HLLSet → materialize → tokens'
```

Round-trip law. Through the door and back, the sketch is recovered **exactly** — materialization emits only tokens whose bits are set in the source, so the traveller may open the door as often as needed without degrading the map.

> The context window becomes a **managed working set** — unpacked from the lattice on demand — instead of a buffer that grows with everything ever said.

MANAGE BEFORE YOU CROSS

`shrink — focus`

Meet the context with the query (H_ctx ∩ Q), restrict to a sub-lattice, or take a minimal cover.

`expand — gather`

Join neighbouring nodes, walk further into the commit DAG, lower the threshold, add views.

`restore — cross`

Materialize the result to tokens, let the host LLM move inside, re-ingest the output to advance the map.

All three are lattice operations — done **before** materializing, so the token context is a design choice, not a by-product of sequence length.

One loop: map → door → move → door → map

EWM lattice
the map — exact, persistent waypoints

→

materialize
the door — unpack detail to tokens

→

LLM attention
the locomotion — graduated motion in detail

→

ingest
the door back — re-compress

← a new node is committed; the commit advances the map ←

The lattice chooses the waypoint; the door unpacks it; attention makes the move inside the unpacked detail; ingestion re-compresses the result; the commit advances the map.

The support–union lemma is the exact seam between the two halves: what crosses the door is always a lattice node on one side and a token context on the other.

> The map prunes the key set **before** the motion begins — attention runs where it matters.

WHY THE LOOP IS SAFE

Round-trip law — crossing the door and back recovers the sketch exactly.

Idempotent joins — re-running a turn yields the same node; recovery is a pop, not a rebuild.

Content addressing — every node on the way is named by its digest and auditable.

The win-win: the map has no motor, the motor has no map

	THE MAP — LATTICE	THE LOCOMOTION — ATTENTION
gives	exact waypoints; durable, auditable memory; controlled shrink/expand; collision groups	differentiable travel; detail resolution; generation of the next step
needs	a motor — it can plan exactly but cannot generate	a map — it generates fluently but has no exact, persistent location
horizon	long, discrete, content-addressed	local, continuous, contextual
state	bitmap $S \subseteq$ plane — monotone, exact	convex coordinates $x \in \mathbb{R}^d$ — reversible, smooth
failure mode	collision ambiguity at scale	context blow-up; drift; no durable state

> Each layer is precisely what the other lacks. The integration is not a replacement of either paradigm — **it is the closure of both**.

Every transformer device has a lattice counterpart

TRANSFORMER	EWM LATTICE
query Q	- ingested query HLLSet
key K_i	- stored node — content address h:<sha1>
value V_i	- materialised tokens $M(K_i \cap Q)$
score q·k	- overlap form ⟨x,y⟩ under a learned metric
softmax weights	- free-energy minimiser — smoothed threshold
supp(output)	- traversal join $\cup \{ V_i : \text{score} \geq \theta \}$
multi-head	- a family of worlds — one lattice, many readings
output projection W_O	- the bridge between worlds
depth / early exit	- read only what is new; stop when nothing new remains
causal mask	- the monotone ring basis of the committed prefix
position	- trajectory frame; rotating coordinates
KV cache	- the commit DAG; the running tip
residual stream	- union accumulation; D/R/N per step
Q/K/V projections	- bootstrap views — Ashby's variety law
RAG	- set-key traversal — the map lookup
flash-attention tiling	- a traversal order over the key set

> An identification of **roles** — explanatory and constructive, not an equivalence of trained weights.

Three modes — three points on the temperature axis

MODE A — address-key

A token's key is its bit address, embedded by a learned table. Tokens in the same cell share a key; values disambiguate. The lattice supplies the partition; learning supplies the metric. Reformer-style hashing is the frozen limit.

the relaxed end — warm

MODE B — set-key

Query and keys are HLLSets; the score is containment or Jaccard; the value is the materialised overlap. No dot product over learned vectors — a lattice morphism and a LUT lookup. This is the map lookup; RAG is its special case.

the exact end — zero temperature

MODE C — hybrid

The learned dot product plus a lattice similarity term with a scheduled weight λ. The dot product remains the workhorse; the lattice term steers it toward the selected waypoint — and its contribution is measurable by ablation.

steered relaxation — between

THE PRICE LIST OF EXACTNESS

Exactness is checked by a **coverage invariant**: the LUT must resolve every active bit. Collisions are the only inexactness — pairwise probability 1/3072 — repaired by a second seed at ~10^{-7} and a third at ~10^{-11}.

The dense end pays in quadratic compute instead; **the temperature axis is also a cost axis**.

TRAVERSAL SIGNALS — MEASURED

On a synthetic 10×10 scene clip: all 9 scene cuts detected by the residual; bit novelty separates cuts from drift by 3.75×; 534 coordinate re-pivots over 100 commits.

Novelty, surprise, representability, re-basing — the natural triggers for opening the door.

The unanswered question: who should move?

The state machine gave a model a state; the seam joined map and motion. What was never learned is **who should move**: when materialized tokens must become real tokens again, *which host LLM* — with which encoding — should restore them?

The Emerging LLM (eLLM) answers with a **dispatcher** that learns a routing policy from the lattice itself: given the lattice state and the materialized tokens, choose the guide — the host — whose encoding restores the tokens best.

> **Host-agnostic by construction.** We do not train tokenizers, and we do not replace the hosts. Host encodings are the *languages* the dispatcher speaks — a new host means learning a new language, not changing the lattice.

The dispatcher keeps **no registry of available LLMs**. Each host owns its own trajectory state H(t) = (S(t), H(t−1), D, R, N); the **top HLLSet** — the collection of all hosts' states — is the implicit choice set, recomputed whenever any host's state changes.

POSITIONING

win-win with every host LLM

Hosts supply encodings and restore real tokens; the dispatcher learns their languages and sends them work.

coexistence, not replacement

Existing routers (Jev, Laya, structural) run behind the same seam — as baselines, validation labels, or fallbacks.

context, not tokens

The lattice is about context: two hosts whose contexts coincide on the lattice produce semantically similar output — so hosts must be contextually different, and learning happens where contexts cross borders.

The UI is a **proxy** in front of a host's prompt: it intercepts the host's encodings and redirects them to the dispatcher and the state machine.

Two processes, one store, one channel

SERVER — the dispatcher

Serves requests; reads the committed policy; refreshes it between requests. Never schedules, never blocks, never waits on the learner.

↓ read tip

TRAINER — the policy learner

Owns the scheduler; samples the routing logs; learns continuously; publishes only on commit — postponed to slow time.

↓ commit

THE STORE — THE STATE MACHINE'S MEMORY PATTERN

- **WIP** — in-memory, per-process (the live window)
- **cache** — shared across processes
- **persisted** — the content-addressed commit DAG; the tip is the head. Serving uses the persisted layer; WIP publishes only on commit.

THE INVARIANT

> **The only channel between trainer and server is a commit.** Because every artifact is idempotent, immutable and content-addressed, the hand-off needs no lock, no handshake and no scheduler — a torn or partial state is not representable. One trainer by default; N trainers add no work to the dispatch path.

A window: six steps, transactional by design

1	`schedule`	Idle / sleep / manual — the scheduler decides when a learning window opens; the server is never involved.
2	`sample`	Draw routing samples from the lattice and the logs: (state, options, decision, confidence).
3	`state`	Through the door: materialized tokens plus each host's codebook gate.
4	`learn`	One policy step per batch — a distilled typed head, or a gradient-free rearrangement of dispatch ranks.
5	`guard`	Non-empty · no regression on held-out probes · codebook coverage · minimum gain — any veto rolls the window back.
6	`commit`	Accepted windows accumulate; the commit is postponed to slow time, publishing many windows in one atomic tip move.

> **Every window is transactional:** a vetoed window changes nothing, and an accepted one becomes one content-addressed checkpoint on the chain. Policy learning *is* a commit DAG — recovery is reading the tip, rollback is a pointer move.

The teacher is the lattice's own reading

A routing sample is not a (query, document) pair but a tuple **(state, options, decision, confidence)**. The decision is the **native ring-basis label**: the host whose codebook gate covers the materialized context best, read off the evolving basis.

No external LLM teacher. The label **emerges with the lattice**, is independent of any host's own labels, and is traceable through basis generations. External router logs are validation labels, never the teacher of truth.

```
coverage c  — how much of the host's gate covers the tokens
novelty  N  = |tokens \ gate|
retained R  = |tokens ∩ gate|
surprise r  = |residual| vs the ring span
```

> Every quantity the curriculum needs was **already computed** by the state machine — the Emerging LLM consumes the trajectory readings as a curriculum instead of as diagnostics.

SAMPLING STRATEGIES

`RecentFirst — log order`

train on the latest routing steps

`NoveltyWeighted — N + 1`

prefer dispatches with uncovered tokens

`ConfidenceWeighted — 1 - conf`

prefer steps where the label is unsure

`BalancedDnr — (N+1)/(R+1)`

new material still anchored in the gate

Bootstrap views (Ashby): the same routing stream measured at several n-gram orders — views are multiplied, not the dispatcher.

The demo: three guides, one travel coordinator

On a single RTX 3060, the notebook hires **three local guides** — DeepSeek-R1-Distill-Qwen-1.5B, Qwen2.5-1.5B-Instruct, gpt2 — each speaking its own token language. A **travel coordinator** (the dispatcher) learns which guide to send along for each leg, and the whole trip takes about a minute.

First every guide walks every route; then the map's own readings (the ring-basis teacher) name the best guide per leg; a small apprentice head distils that judgment, takes over the dispatch desk, and **keeps learning while trips run**.

```
every guide walks every route  (open loop)
→ the map names the best guide  (teacher loop)
→ distil an apprentice → serve  (dispatch desk)
→ keep learning on the road  (online loop)
```

> The demo's purpose is not the numbers — it is **to surface the hidden problems** (right) while the loop is still cheap to change.

WHAT THE JOURNEY EXPOSED

Distribution shift — every guide walks during training; only the chosen one walks afterward.

Cold start — on day one the map is blank; the apprentice's first dispatch is a guess.

Overfit on tiny logs — twelve trips memorised is not a policy: held-out probes fall to chance.

Online collapse — the apprentice plays it safe and starts sending the same guide everywhere.

Identity check — the round-trip through the door must be re-verified for each guide's real token order.

Each one is now a named item on the roadmap — replay, regularisation, better exploration.

THE MAP · THE MOTION · THE DOOR · THE CHOICE

The pair is the complete primitive.

One seam — the support of attention is the lattice join: the bitmap decides the working set, the weights move inside it.

One knob — threshold and temperature are two faces of the same selection rule; the softmax is the annealing schedule.

One loop — map, door, move, door, map: every step is exact, auditable, and safe to replay.

One dispatcher — learning who should move, taught by the lattice's own reading, behind a guard-railed commit chain.

WHAT IS NEXT

Surprise-gated door — open materialization only when a traversal signal fires.

Gradient-free policy — the rank-lattice path: rearrange the registry, update no weights.

The agnosticism experiment — add a new host encoding; show the dispatch adjusts, lattice unchanged.

The full sheaf — the world family as a site; the correspondence is its finite stage.

Acknowledgment — this deck was prepared with the assistance of Kimi (Moonshot AI).

$$\mathrm{supp}\left(\sum \alpha_i \cdot V_i\right) = \bigcup \{ V_i : \alpha_i > 0 \}$$

github.com/alexmy21/ewm-state-machine